Agentic Text-to-SQL

Final Project Report
Authors: Nick Labuda & Rahul Muthuswamy

Abstract

This report presents our re-implementation and findings of an agentic framework, ReFoRCE, for translating natural language questions into executable SQL queries over extremely complex database schemas. We evaluate our approach on the Spider-2-Snow benchmark, comparing against a baseline Spider-Agent. ReFoRCE demonstrates a relative improvement of approximately 8% in exact-match SQL accuracy, which we match with a 10% improvement in our testing. We also outline future enhancements to further increase query precision and system scalability.

1. Introduction

Natural language interfaces for databases lower the technical barrier to data analytics, enabling non-technical users to interact with structured data through everyday language. Text-to-SQL models aim to automatically convert natural language inputs into valid SQL queries. However, in real-world enterprise settings, databases often consist of hundreds of tables and thousands of columns, which exceeds the context-processing capabilities of most current large language models (LLMs). This complexity motivates a more structured, agentic approach to query generation and refinement.

2. Problem Statement

1. **Large & Complex Schemas:** Enterprise databases contain extensive schemas that are difficult to interpret and navigate, even for seasoned developers.

2. **LLM Context Limitations:** The limited context window of LLMs makes it difficult to ingest and reason over large schemas, often resulting in incomplete or incorrect SQL queries.

3. Benchmark

We evaluate our system using the Spider-2-Snow benchmark, an extension of the original Spider 1.0 dataset (2018):

- **Size:** 547 samples.
- **Input Data:** Each sample includes a Markdown schema summary, DDL statements, and JSON-formatted metadata (tables, columns, types, descriptions, and sample rows).
- **Evaluation Method:** Generated SQL queries are scored by logical equivalence and execution correctness against expected answers.

On Spider 1.0 dev, the first benchmark that was released, DAIL-SQL with GPT-4 got 86.6% of the problems correct. On the latest, Spider-2-Snow, the framework achieves 2.2% of the problems correct. The creators of the Spider Benchmarks then created their own framework, which they call Spider Agent. It achieves 24.5% on Spider-2 with the best current model, Claude 3.7 Sonnet. The baseline Spider Agent is explained below.

4. Baseline: Spider-Agent

The baseline system, Spider-Agent, uses an LLM to iteratively complete SQL tasks through a structured ReACT (Reasoning + Action + Observation) loop. Despite its agentic capabilities, it only generates an actual SQL for 30% of the samples. None of the SQL queries were correct either upon execution. We implemented the Spider Agent and ReFoRCE Agent for the QwQ model. The overall accuracy on Spider-2-Snow is approximately 8.96% with QwQ.

5. Proposed Method: ReFoRCE

ReFoRCE builds on Spider-Agent by introducing structured mechanisms to guide reasoning, correct errors, and manage schema complexity.

5.1 Schema Compression

To address the challenge of fitting extensive schema information into limited context windows for large language models, ReFoRCE introduces a pattern-based schema compression technique. This method systematically analyzes the structure and semantics of database schemas to identify groups of similar tables—those that share common patterns in naming conventions, column types, and relationships. Once identified, these groups are compressed by replacing individual table definitions with a single, representative DDL (Data Definition Language) statement that

captures the shared structure and intent of the group. This approach drastically reduces redundancy while preserving essential information.

By condensing repetitive schema patterns, ReFoRCE can reduce the overall input length by up to 90%. This compression makes it feasible to include a high-level overview of even large-scale schemas within the token constraints of modern LLMs. The resulting compressed schema string is both compact and informative, retaining enough structure and semantics to support accurate downstream tasks such as SQL generation, column linking, and question answering. This compression approach enables efficient and scalable reasoning over complex database systems without sacrificing critical contextual understanding.

5.2 Format Specification

For each natural language query, a format specification is generated that outlines the expected structure of the SQL output—specifically, the required output columns along with their data types and the anticipated number of rows (such as a single row for superlative queries). This specification acts as a structural blueprint, guiding SQL generation to align with the query's intent and enabling downstream validation of the query's correctness based on the actual results returned.

5.3 Column Exploration

To identify and extract relevant schema information for SQL generation, a secondary LLM session is invoked to run exploratory prompts targeting potentially useful tables and columns. These prompts aim to retrieve representative data samples—such as distinct column values, row patterns, or joined outputs—that help the model better understand the underlying data distribution and relationships. By proactively surfacing schema characteristics through example-driven exploration, this process supplements schema compression with real, contextual evidence drawn directly from the database.

To ensure reliability and robustness, each exploration prompt undergoes validation through Algorithm 1, a self-correcting execution framework. This algorithm sequentially pops and executes each SQL query, saving the result only if it is syntactically valid and returns non-empty output. In the event of errors or empty results, the system initiates up to three rounds of LLM-guided self-correction, refining the original prompt based on feedback. If a correction proves successful, it is generalized to improve the remaining queries in the batch. The process terminates early if five consecutive queries fail, preventing unnecessary computation. The final output is a dictionary mapping each corrected exploration prompt to its validated result—effectively bootstrapping a reliable set of contextual insights for downstream SQL generation.

5.4 Self-Refinement (Algorithm 2)

With the compressed schema, format specification, and validated exploration results in hand, the LLM proceeds to generate candidate SQL queries in an iterative manner—allowing up to five attempts per query. After each generation, the query is executed and its output is validated against the predefined format specification, ensuring the results align with both the expected structure and semantics of the original natural language question. This process is designed to maximize both precision and resilience in SQL generation, particularly in complex or ambiguous cases.

Throughout this iterative phase, the system actively monitors the consistency and correctness of outputs. If at least two generations produce identical and format-valid results, the corresponding SQL query is accepted as the final output. In cases where errors arise—such as syntax issues or deviations from the expected format—the LLM is prompted to refine the query, with each error type receiving up to three self-correction attempts. This multi-round, validation-driven process ensures that the accepted SQL not only executes successfully but faithfully reflects the user's intent as encoded in the format specification and schema context.

6. Experimental Results

A subset of 20 random Spider-2-Snow samples was selected for evaluation due to runtime constraints. Results are summarized below:

Method	Generation Rate	Exact-Match Accuracy
Baseline Spider-Agent (QwQ-32B)	30%	0%
ReFoRCE (QwQ-32B)	90%	10%

ReFoRCE demonstrates a large increase in SQL generation success and an important improvement in query accuracy. These results are in line with the results of the ReFoRCE paper itself, which boasts a 7.68% increase over Spider-Agent (using o1-preview).

7. Discussion

ReFoRCE substantially improves SQL generation coverage over the Spider-Agent baseline. Although absolute accuracy remains limited, the results highlight the potential of modular, agentic techniques in handling complex schema reasoning. The architecture of schema

compression, structured exploration, and iterative refinement offers promising opportunities for further improvement.

8. Future Directions

We identify several areas for continued enhancement:

8.1 Additional ReFoRCE Improvements

These are mentioned in the ReFoRCE paper but were not implemented due to either lack of explanation or time/resources.

- **Parallelization with Voting:** Run self-refinement in parallel, selecting the best query through majority consensus.
- **CTE-Based Refinement:** Break queries into modular components using Common Table Expressions.
- **Cross-Database Support:** Extend the framework to work across database dialects beyond Snowflake.
- **Expanded Exploration:** Generate a broader variety of exploration queries (>10).
- **Improved Error Tolerance:** Allow more retries and handle additional error scenarios (>3).

8.2 Retrieval-Augmented Schema Annotation

To enhance schema compression using Retrieval-Augmented Generation (RAG), we would leverage both the original Markdown and JSON documentation as contextual input. These documents often contain critical domain-specific knowledge—such as table descriptions, column definitions, and usage examples—that are typically lost during standard compression processes. By using a RAG pipeline, we retrieve relevant documentation snippets in response to prompts about individual columns or tables, allowing the system to generate informed summaries and annotations tailored to the specific schema elements.

Then we would generate domain-specific annotations based on the retrieved context. These annotations are crafted to retain the intent and semantics of the original schema elements, after compression. This enriched schema—now augmented with context-aware, domain-specific annotations—provides a more semantically meaningful input for better results in SQL LLMs. As

a result, models can generate more accurate and context-sensitive queries, especially when dealing with ambiguous or abbreviated column and table names in the compressed format.

This pipeline could prove to be very beneficial in ReFoRCE. Especially with column exploration, as the passed in schema now has domain aware annotations to better understand which tables to explore more based on the prompt.

9. Conclusion

ReFoRCE introduces a robust, agentic framework for translating natural language tasks into accurate SQL queries over complex database schemas. We have validated its effectiveness on a subset of the benchmark through our re-implementation. By compressing schemas, exploring data semantics, and refining SQL outputs iteratively, ReFoRCE shows measurable improvements over existing baselines. Its modular design offers flexibility and extensibility, establishing a foundation for future innovation in LLM-based data querying systems.

10. Project Files

Link to Repo: https://github.com/nic3245/ReFORCE
ReForCe Agent Impl.: methods/spider-agent-snow/spider_agent/agent/reforce_agent.py
Copy the emailed snowflake_credential.json file to:
methods/spider-agent-snow/snowflake_credential.json
Add a logs/ folder at: methods/spider-agent-snow/logs/
Then follow README.md at: methods/spider-agent-snow/README.md